



Git – Introduction

Module 7

Git setup

Scenario overview

- You are tired of having different versions of the same scripts running in your environment.
- You want to be able to leverage version control within your script repository.
- Your engineers are confused on updates and which version of a specific script or file they are supposed to be used.
- You want control over your scripts used in the production environment.

What is DevOps?

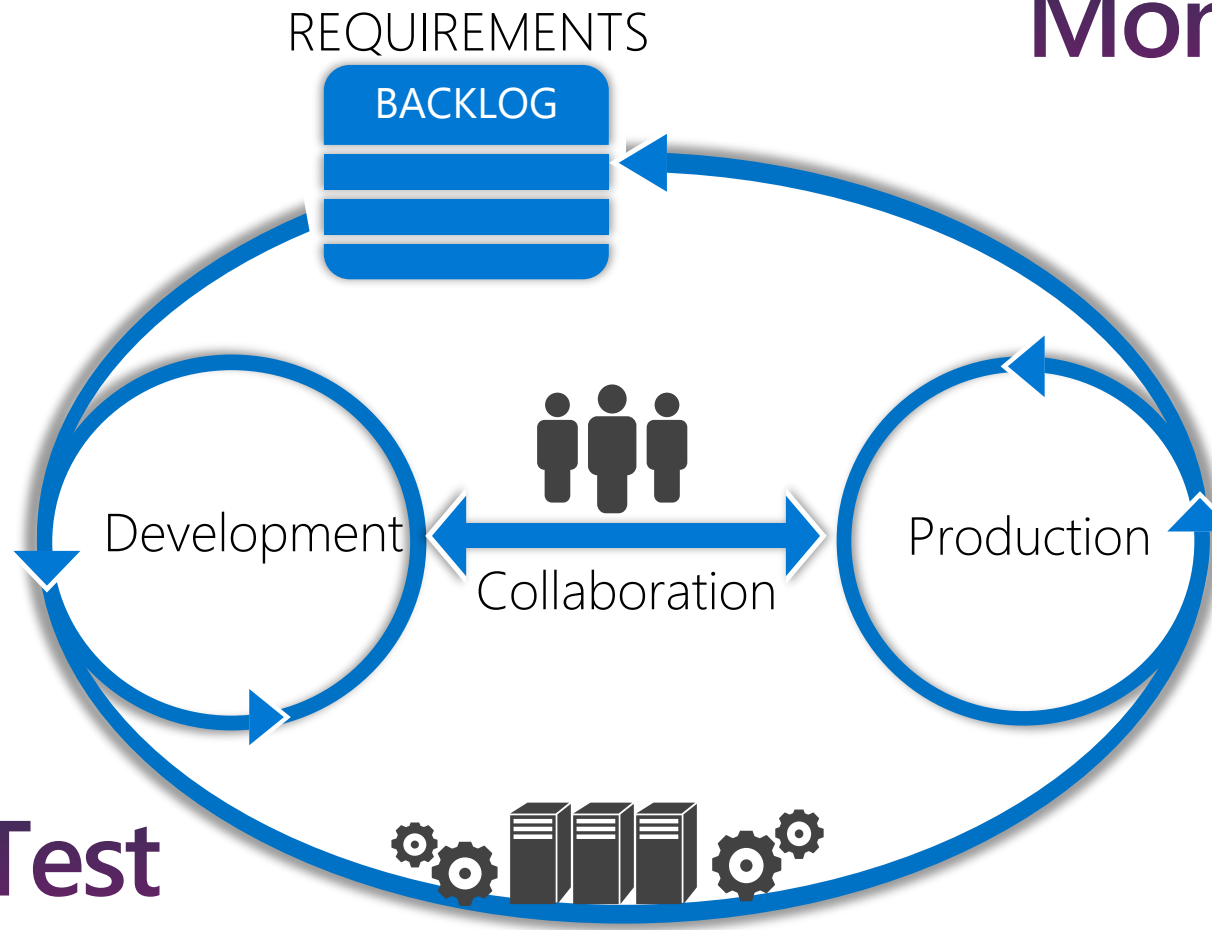
- The union of people, process, and products to enable continuous delivery of value to your end users.
- The contraction of “Dev” and “Ops” refers to replacing siloed Development and Operations to create multidisciplinary teams that work together with shared and efficient practices and tools.



DevOps – Deliver Faster, Smarter, and Continuously

Plan

Monitor + Learn



Develop + Test

Release

Introduction To Version / Source Control

Version control of
scripts, files,
applications, etc.

Records changes
to a file or set of
files over time

Allows file /
project versioning

Local or
centralized

Distributed version
control change
repository option

Easily swap
versions

Ideal for teams

Undo changes

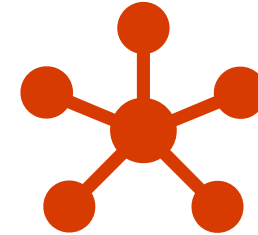
Stored on each
machine

Types of Version Control



Distributed

- Everyone has a copy
- Changes made locally
- Merged with an online central copy
- Doesn't generally require connectivity



Centralized

- Master copy is stored centrally
- Requires network connectivity

CI/CD strategy

Continuous Integration (CI)

- Team develops from same repository
- Limited drift from master branch
- Results contain few bugs
- Software works properly

Continuous Delivery (CD)

- Produce software/updates in short cycles
- Allows: building, testing, and releasing code with greater speed and frequency
- Reduces: cost, time, and risk of delivering changes by allowing for more incremental updates to applications/scripts

Source Control Management



Harness collaboration



Enable parallel development



Minimize integration debt



Act as a quality gate

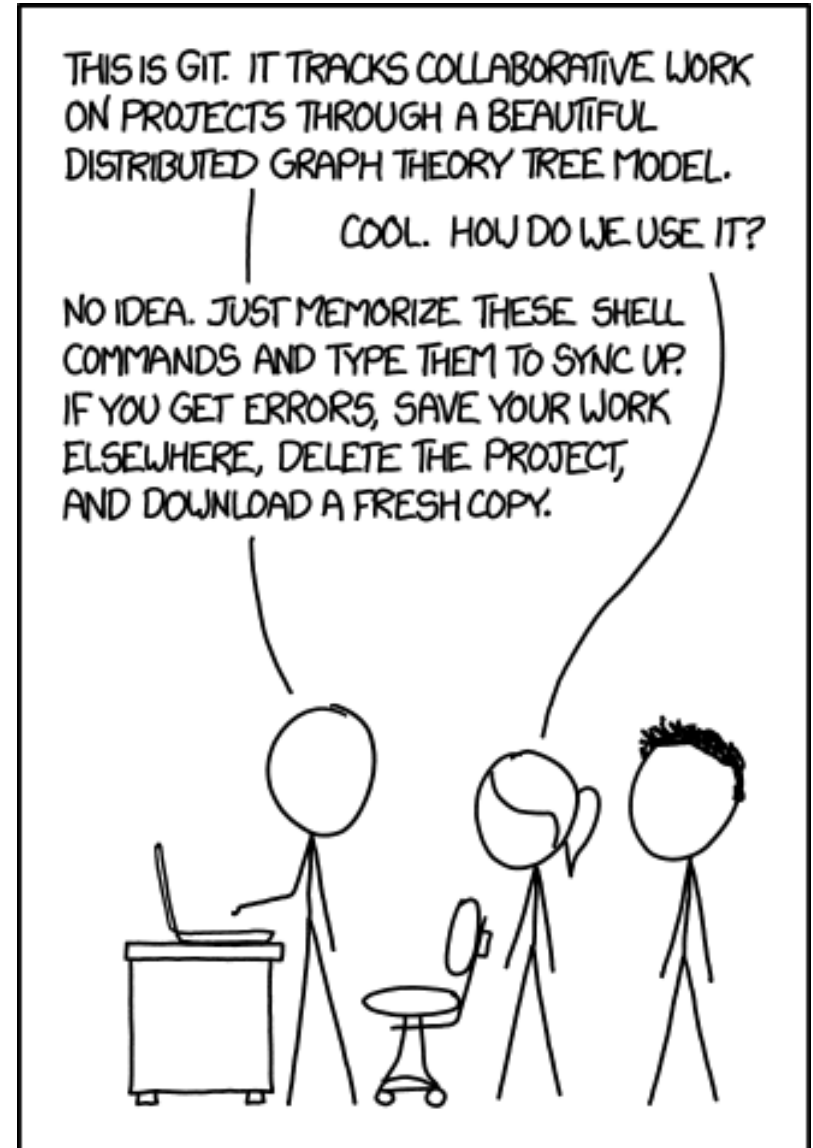
What is Git?

- Distributed version control system
- Created in 2005 by Linus Torvalds
- Lightweight and fast
- Supports parallel development (branching)
- Fully distributed
- Open Source
- Integrated with many Content Index (CI) tools



Challenges of Git

- New for many operational engineers
- Command line based
- What's the point?
- Afraid to use Git
 - New toolset
 - Very powerful ("Dangerous")
 - Not user friendly
 - I don't want to become a developer



Git Commits



Retains each version by tracking ALL changes



Displays exact changes



Undo is holistic and reverts

Git Setup

Clone

- Create a new local repository to clone data into
- Copy existing repository to clone location
- Adds .git folder
- Adds remote origin
- Run with: *git clone ssh_url*

```
PS c:\> git clone https://github.com/anwaterh/MyNewRepository.git  
Cloning into 'MyNewRepository'...
```

Git Structure



A Git repository is created by using `git init`



Git creates a `.git` folder



`.git` folder contains information needed for Git to function

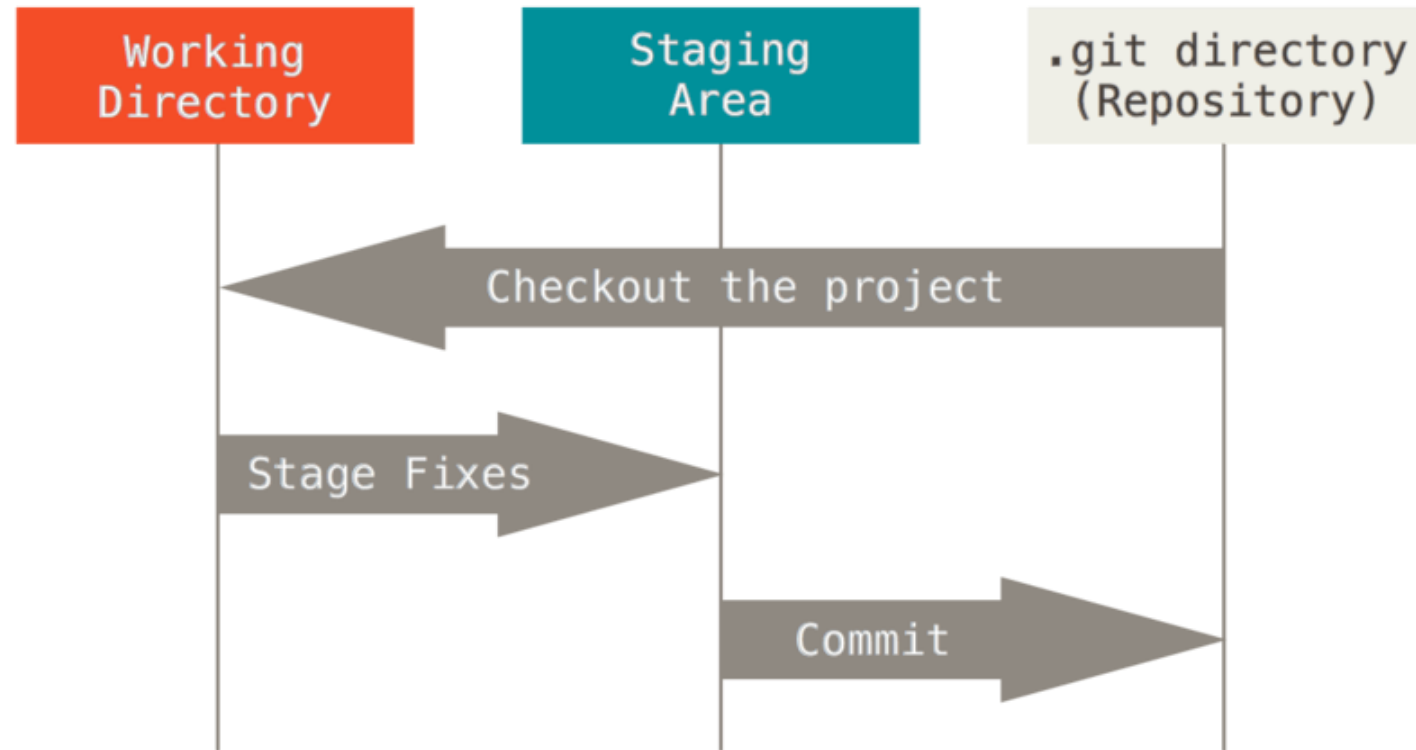


To remove Git, remove the `.git` folder and retain all project files

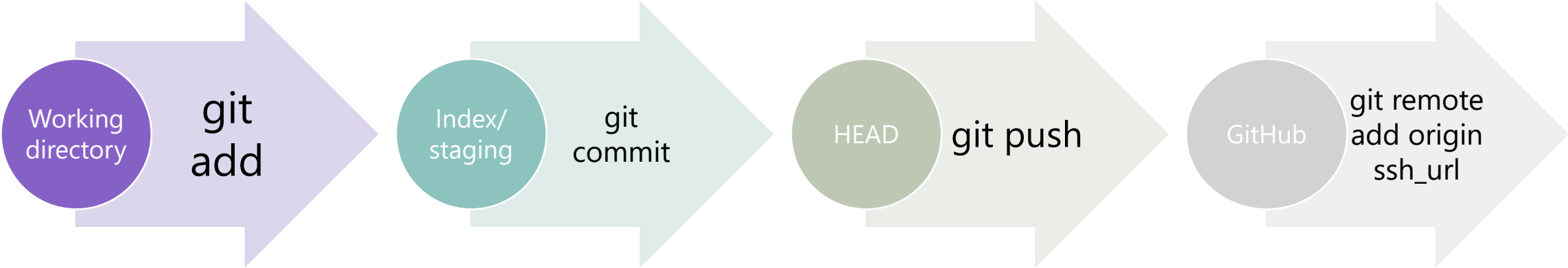
Git Layers

Three stages to track changes:

1. Working directory
2. Index/staging
3. HEAD



Git Process



Logging into a Git Repository

Repositories



Can be either hosted online or local (GitHub, GitHub Enterprise)



Represents the main project



Contains: files, commits, branches, and history for the entire project



Best practices:

Maintain a single source repository

Place all needed content for a project in a single repository

Minimize branches

Git vs. GitHub

- **Git**: a revision control system, a tool to manage your source code history
- **GitHub**: a hosting service for Git repositories, only stores code



Create Repository

- **Create New** – *git init <RepositoryName>*

```
PS c:\> git init MyNewRepository  
Initialized empty Git repository in C:/MyNewRepository/.git/
```

- **Clone from another location** – *git clone <location .git>*

```
PS c:\> git clone https://github.com/anwaterh/MyNewRepository.git  
Cloning into 'MyNewRepository'...
```

- A local copy of source files is available

Name	Date modified	Type	Size
 .git	4/10/2018 4:04 PM	File folder	

Workspace / Staging Area

- Files are created / edited in your workspace
- To track a file it first must be added to the staging area
- Staged files can then be committed (snapshot of the current staging area)
- All files / changes can be added – or only individual files



Adding Content to Staging Area

- **Add a file to staging area** – *git add <filename>*

```
PS c:\MyNewRepository> git add .\script.txt
PS c:\MyNewRepository> git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)

        new file:   script.txt
```

- **Add all files to staging area** – *git add .*

Removing Staging Area Content

- Remove a file from staging (won't be part of the commit)

```
PS c:\MyNewRepository> git rm .\script.txt -cached
```

```
Rm 'script.txt'
```

```
PS c:\MyNewRepository> git status
```

```
On branch master
```

```
No commits yet
```

```
Untracked files:
```

```
(use "git add <file>..." to include in what will be committed)
```

```
    script.txt
```

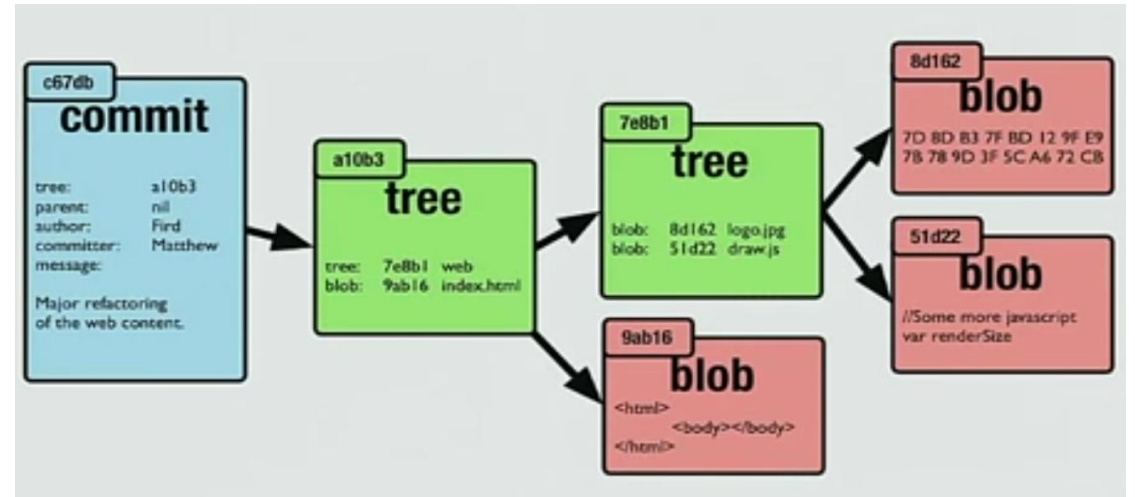
```
Nothing added to commit but untracked files present (use "git add" to track)
```

Commit

- Snapshots the repository at that point
- All staged files become part of the commit
- Saves email address and username
- Contains a message describing the changes
- Commit has a unique hash which is used as a reference
- Use a text editor or the `-m` switch to provide the message

Commit - Under the Hood

- Commit stores the entire blob, not just deltas as hashes: 530a6ac7c3682458ec6307a7c2c350ed849bfaf7
- Not duplicating content – leverage same blob
- Multiple commits can point to the same blob content
- Commit points to a tree, then to a blob
- Object DB used
- Start with root tree
- Updates parent
- .git folder to start



Committing Changes

- **Commit all changes** – *git commit -m "<commit message>"*

```
PS c:\MyNewRepository> git commit -m "My commit"
[master 67b2b30] My commit
Date: Tue Apr 10 16:21:50 2018 +1000
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 script.txt
PS c:\MyNewRepository>
```

- Use the **-amend** switch to change the last commit

```
PS c:\MyNewRepository> git commit -m "Altered commit" --amend
[master 67b2b30] Altered commit
Date: Tue Apr 10 16:23:50 2018 +1000
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 script.txt
PS c:\MyNewRepository>
```

Viewing History

Git log

```
PS c:\MyNewRepository> git log
Commit ebf96ff9862bc450a4b61c48742c0ec64639d1c (HEAD -> master)
Author: Anthony Watherston <anwather@contoso.com>
Date: Tue Apr 10 16:25:40 2018 +1000
```

Made a small change

```
Commit 67b2b30e8cc6b8167ba947a55440730951ac44bq
Author: Anthony Watherston <anwather@contoso.com>
Date: Tue Apr 10 16:25:40 2018 +1000
```

Altered commit

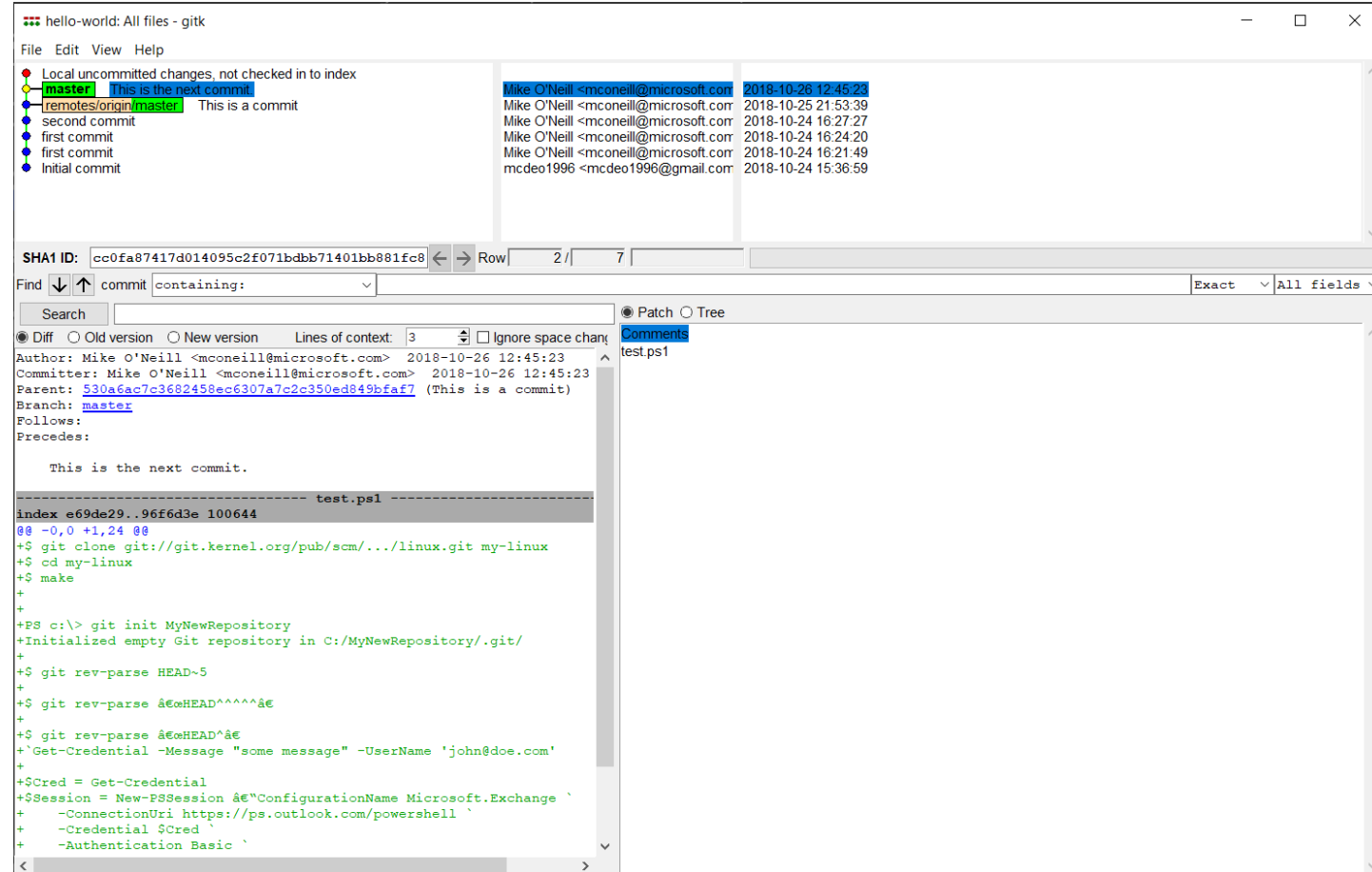
History Options

- Many options to condense output and add decorations

```
PS c:\MyNewRepository> git log --oneline
ebf96ff (HEAD -> master) Made a small change
Commit 67b2b30 Altered commit
PS c:\MyNewRepository> git log --oneline --graph
* ebf96ff (HEAD -> master) Made a small change
* Commit 67b2b30 Altered commit
PS c:\MyNewRepository>
```

History Options Cont.

Use **gitk** for a GUI history



Demonstration

Advanced Features



Git – Introduction

(30 minutes)

- Installing Git
- Setting up first repository

